FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Trabajo Práctico Nº1: File Transfer

[TA048] Redes
Curso Alvarez Hamelin
Fecha de Entrega: 8/10/2026

Alumno	Número de padrón	Email
De Nobili, Lautaro	107394	ldenobili@fi.uba.ar
Kwiatkowski, Lorenzo	110239	lkwiatkowski@fi.uba.ar
Kotelchuk, Camila	110289	ckotelchuk@fi.uba.ar

Índice

1. Introducción **2**

2. Hipótesis y suposiciones realizadas **2**
2.1. Hipótesis . . . 2
2.2. Suposiciones . . . 2

3. Implementación **2**
3.1. Estructura del código . . . 2
3.2. Servidor concurrente . . . 3
3.3. Un mismo protocolo para cliente y servidor . . . 3
3.4. Flujo . . . 3
3.4.1. Upload . . . 3
3.4.2. Download . . . 3

4. Pruebas **4**
4.1. Transferencia con pérdidas . . . 4
4.2. Comparación de tiempos entre protocolos . . . 4
4.3. Archivos binarios . . . 5
4.4. Captura de tráfico . . . 5
4.5. Concurrencia . . . 6

5. Preguntas a responder **7**
5.1. Describa la arquitectura Cliente-Servidor . . . 7
5.1.1. En nuestro trabajo . . . 7
5.1.2. Servidor concurrente . . . 8
5.2. ¿Cuál es la función de un protocolo de capa de aplicación? . . . 8
5.2.1. En nuestro trabajo . . . 8
5.3. Detalle el protocolo de aplicación desarrollado en este trabajo . . . 9
5.3.1. Formato de los paquetes . . . 9
5.3.2. Pedido . . . 9
5.3.3. Handshake de 4 mensajes . . . 9
5.3.4. Transferencia de datos . . . 10
5.3.5. Finalización . . . 10
5.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno? . . . 11
5.4.1. Servicios comunes . . . 11
5.4.2. TCP (Transmission Control Protocol) . . . 11
5.4.3. UDP (User Datagram Protocol) . . . 12
5.4.4. Cuándo usar cada uno . . . 12
5.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión; en caso de tenerlos, descríbalos . . . 12
5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo? . . . 12

6. Dificultades encontradas **13**

7. Conclusión **13**

1. Introducción

El objetivo de este trabajo práctico es desarrollar una aplicación de red cliente-servidor para la transferencia de archivos, utilizando la interfaz de sockets de Python. La aplicación ofrece dos operaciones: upload que envía un archivo del cliente al servidor para que este lo almacene y download que obtiene del servidor un archivo previamente almacenado.

La comunicación se realiza sobre UDP, un protocolo de transporte que no garantiza la entrega de los datagramas: se pueden perder, duplicar o llegar desordenados. Es por esto que debemos resolver la confiabilidad de la transferencia en la capa de aplicación. Para ello diseñamos un protocolo propio con un encabezado de tipo de mensaje y número de secuencia, un handshake de establecimiento y un mecanismo de cierre, e implementamos dos estrategias de recuperación de errores: Stop & Wait y Selective Repeat.

El servidor es concurrente: atiende a múltiples clientes a la vez, cada uno con el protocolo que elija. La aplicación se probó con una red simulada con Mininet, forzando pérdidas de paquetes del 10 % y distintos valores de RTT, y se verificó que los archivos transferidos llegan completos y sin cambios.

2. Hipótesis y suposiciones realizadas

2.1. Hipótesis

- La pérdida de paquetes es de hasta 10 % en cada sentido y el RTT es de hasta 300ms, tal como indica el enunciado.
- Los paquetes corruptos son descartados por el checksum de UDP, por lo que desde el punto de vista de la aplicación una corrupción equivale a una pérdida.
- Los paquetes pueden llegar duplicados o fuera de orden; el protocolo lo contempla mediante los números de secuencia.
- Si el otro extremo deja de responder durante un tiempo prolongado (5s para el receptor, 10s para el emisor), se asume que abandonó la transferencia.

2.2. Suposiciones

- Una operación por cliente: cada ejecución de upload o download realiza una sola transferencia.
- Tamaño de payload de 2900 bytes: se eligió para reducir la cantidad de paquetes y de ACKs.
- Ventana de 8 paquetes en Selective Repeat: se eligió una potencia de 2. Con una pérdida del 10 % en una ventana de 8 se pierden en promedio 0,8 paquetes (menos de uno por ventana). Además, con un RTT de 40ms, 8 paquetes de 2900 bytes por RTT permiten un throughput de unos 580KB/s, suficiente para transferir 5MB por debajo del límite de 2 minutos.
- Timeouts fijos de 70ms para Stop & Wait y 80ms para Selective Repeat, pensados para el RTT de 40ms del escenario de prueba.
- El archivo se mantiene completo en memoria durante la transferencia y se escribe en disco al recibir el FIN.

3. Implementación

3.1. Estructura del código

La aplicación está escrita en Python usando la librería estándar. Está dividida en cuatro módulos:

- `common/`: lo que comparten cliente y servidor (constantes, armado y lectura de paquetes, lectura y escritura de archivos, armado del pedido y argumentos de línea de comandos)
- `client/`: el socket del cliente y el handshake de cada operación.
- `server/`: el socket del servidor, un hilo por cliente y la lógica del servidor para cada operación.

- `protocol/`: la transferencia confiable.

Cliente y servidor crean un socket UDP. Solo el servidor hace `bind` a una dirección fija, pero el cliente no lo necesita porque el sistema operativo le asigna un puerto en el primer envío.

3.2. Servidor concurrente

El servidor usa un único socket y un hilo principal que recibe todos los paquetes. Cada cliente se identifica por su direccion (IP, puerto) y tiene asociado un `ClientHandler`, un hilo con su propia cola.

- Si el paquete viene de una dirección nueva, el hilo principal crea un `ClientHandler`, le encola el paquete y lo arranca.
- Si viene de un cliente conocido lo encola en la cola de ese cliente.
- Cuando un `ClientHandler` termina, se elimina del diccionario de clientes.

3.3. Un mismo protocolo para cliente y servidor

Los protocolos no saben si se ejecutan en el cliente o en el servidor, reciben un "contexto" que ofrece dos operaciones: `send_packet` y `get_packet(timeout)`. En el cliente el contexto es `Client` que envía y recibe su propio socket. En el servidor es `ClientHandler` que envía por el socket compartido y recibe de su cola. De esta forma, el mismo código de Stop & Wait o Selective Repeat actúa como receptor o emisor (en el upload el emisor es el cliente; en el download es el servidor).

3.4. Flujo

3.4.1. Upload

```
Cliente                                  Servidor
   |--- START (UPLOAD|archivo|sr) ---------->|  _wait_for_start()
   |<------------------------------ ACK ----|
   |--- CONFIRM ---------------------------->|  handshake completo
   |<-------------------------- READY x3 ----|  UploadHandler.execute()
   |                                         |
   |--- DATA (seq 0) ----------------------->|
   |<------------------------ ACK (0) ------|  Stop & Wait o
   |--- DATA (seq 1) ----------------------->|  Selective Repeat
   |<------------------------ ACK (1) ------|
   |                  ...                    |
   |--- FIN ------------------------------->|
   |<--------------------------- FIN_ACK ----|  se guarda el archivo
```

1. El cliente envía START y espera ACK (reintenta cada 2s, hasta 10 veces).
2. Al recibir el ACK, envía CONFIRM y espera READY (reenvía CONFIRM cada 1s, durante hasta 5s).
3. El servidor, al recibir el CONFIRM, envía 3 READY y queda como receptor.
4. El cliente, como emisor, envía los DATA y espera sus ACK según el protocolo elegido.
5. Al terminar, el cliente envía FIN; el servidor responde FIN_ACK y escribe el archivo en el storage.

3.4.2. Download

```
Cliente                                  Servidor
   |--- START (DOWNLOAD|archivo|sw) -------->|  _wait_for_start()
   |<------------------------------ ACK ----|
   |--- CONFIRM ---------------------------->|  DownloadHandler.execute()
   |                                         |  verifica si existe el archivo
   |<-------------------------- ERROR x10 ---|  si NO existe: fin
```

```
|<------------------------- READY x3 ----|  si existe:
|                                        |
|<------------------------ DATA (seq 0) --|
|--- ACK (0) ----------------------------->|  Stop & Wait o
|<------------------------ DATA (seq 1) --|  Selective Repeat
|--- ACK (1) ----------------------------->|
|                  ...                   |
|<--------------------------------- FIN ---|
|--- FIN_ACK ----------------------------->|  el cliente guarda el archivo
```

1. El handshake es igual al del upload.
2. Al recibir el CONFIRM, el servidor verifica si el archivo existe. Si no existe, envía ERROR 10 veces y el cliente informa el error.
3. Si existe, envía 3 READY y queda como emisor; el cliente es el receptor.
4. Al terminar, el servidor envía FIN; el cliente responde FIN_ACK y escribe el archivo en la carpeta correspondiente.

4. Pruebas

Todas las pruebas se realizaron en Mininet con la topología de `src/topo.py`: un servidor (h1, 10.0.0.1) y un cliente (h2, 10.0.0.2) conectados por un switch.

4.1. Transferencia con pérdidas

Se subió el archivo de 5MB con Selective Repeat y 10 % de pérdida. La transferencia terminó correctamente en 32,4s por debajo del límite de 2 minutos y el hash MD5 de la copia coincide con el del original.

```
mininet> h2 time python3 src/upload.py -H 10.0.0.1 -p 5000 -s pruebas -n up5.txt -r sr
INFO - Preparando subida del archivo: up5.txt
INFO - Conexion establecida - Iniciando transferencia de datos...
INFO - Subida del archivo 'up5.txt' finalizada

real    0m32.375s
user    0m0.183s
sys     0m0.117s
mininet> h2 md5sum pruebas/up5.txt storage/up5.txt
3c416ee65da38987e02b206667b33aea  pruebas/up5.txt
3c416ee65da38987e02b206667b33aea  storage/up5.txt
```

Figura 1: Upload de 5MB con Selective Repeat y 10 % de pérdida: salida del cliente con el tiempo y comparación de hashes.

4.2. Comparación de tiempos entre protocolos

Se midió el tiempo de transferencia para tres tamaños de archivo con 0 % y 10 % de pérdida. El throughput se calculó como tamaño del archivo dividido el tiempo.

Archivo	Pérdida	S&W (s)	SR (s)	S&W (KB/s)	SR (KB/s)
1 MB	0 %	14,4	2,1	69	482
2 MB	0 %	28,6	3,8	70	529
5 MB	0 %	71,6	9,1	70	549
1 MB	10 %	23,7	5,8	42	172
2 MB	10 %	49,4	15,4	40	130
5 MB	10 %	122,9	32,4	41	154

Cuadro 1: Tiempos de upload y throughput promedio con RTT de 40 ms.

Selective Repeat fue más rápido en todos los casos. Sin pérdidas, para el archivo de 5MB tardó 9,1s frente a 71,6s de Stop & Wait. Es decir, unas ocho veces menos, que coincide con el tamaño de la ventana: Selective Repeat envía 8 paquetes por RTT y Stop & Wait uno solo.

Con 10 % de pérdida, Selective Repeat tardó 32,4s frente a 122,9s de Stop & Wait, casi 4 veces menos, y cumplió con el requisito de transferir 5MB en menos de 2 minutos

En todas las pruebas, Stop & Wait mantuvo un throughput casi constante para cada nivel de pérdida, independiente del tamaño de archivo, porque envía solo un paquete por vez.

4.3. Archivos binarios

Para verificar que la aplicación transfiere archivos binarios sin alterarlos se descargo una imagen y un PDF almacenados en el servidor. En ambos casos los hashes MD5 coinciden.

```
mininet> h2 python3 src/download.py -H 10.0.0.1 -p 5000 -d descargas -n gatito.jpg -r sr
INFO - Solicitando descarga del archivo: gatito.jpg
INFO - Conexion establecida - Iniciando recepción de datos...
INFO - Descarga del archivo 'gatito.jpg' finalizada - Guardado exitosamente en 'descargas'.
mininet> h2 python3 src/download.py -H 10.0.0.1 -p 5000 -d descargas -n CDN.pdf -r sw
INFO - Solicitando descarga del archivo: CDN.pdf
INFO - Conexion establecida - Iniciando recepción de datos...
INFO - Descarga del archivo 'CDN.pdf' finalizada - Guardado exitosamente en 'descargas'.
mininet> h2 md5sum storage/gatito.jpg descargas/gatito.jpg storage/CDN.pdf descargas/CDN.pdf
cf6d2e5feea8f7accd8e3197fadaf8ab  storage/gatito.jpg
cf6d2e5feea8f7accd8e3197fadaf8ab  descargas/gatito.jpg
3dfc32548086783effa7af84fb365d1a  storage/CDN.pdf
3dfc32548086783effa7af84fb365d1a  descargas/CDN.pdf
```

Figura 2: Transferencia de archivos binarios y comparación de hashes MD5.

Luego intentamos descargar un archivo que no existe en el servidor por lo que este respondió con un mensaje de ERROR y el cliente informó el error sin iniciar la transferencia.

```
mininet> h2 python3 src/download.py -H 10.0.0.1 -p 5000 -d descargas -n no_existe.txt -r sr
INFO - Solicitando descarga del archivo: no_existe.txt
WARNING - Timeout en handshake - Timeout esperando ACK al START - Reintento 1/10
ERROR - El archivo 'no_existe.txt' no existe en el servidor.
```

Figura 3: Descarga de un archivo inexistente: el cliente reintenta el START perdido y luego informa el error.

4.4. Captura de tráfico

Se capturó en la interfaz de h2 una transferencia corta y se analizó con Wireshark.

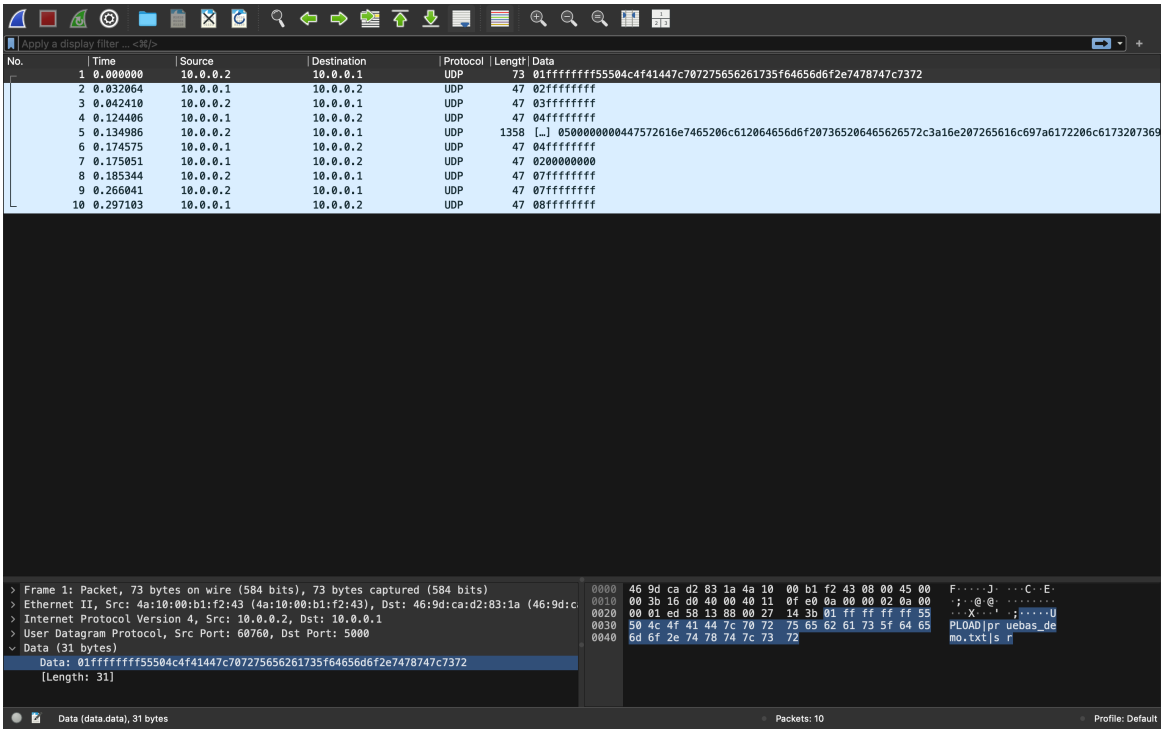


Figura 4: Captura de Wireshark de un upload completo con Selective Repeat y 10 % de pérdida.

4.5. Concurrencia

Para verificar que el servidor atiende a varios clientes en simultáneo se ejecutó `pruebas/prueba_concu.py`, que levanta en Mininet un servidor (h1, 10.0.0.1) y cuatro clientes (h2 a h5) conectados a un mismo switch, con enlaces de 20 ms de delay y 10 % de pérdida. Tras iniciar el servidor, el script lanza en paralelo dos uploads (`up1.txt` de 1MB desde h2 y `up2.txt` de 2MB desde h3) y dos downloads (`down1.txt` de 1MB hacia h4 y `down2.txt` de 2MB hacia h5), todos con Selective Repeat.

En el log del servidor se observa que los cuatro clientes se conectan casi al mismo tiempo, cada uno identificado por su dirección (IP, puerto), y que sus operaciones avanzan intercaladas: mientras se reciben `up1.txt` y `up2.txt`, se envían `down1.txt` y `down2.txt`. Cada cliente se elimina de la lista del servidor al terminar su operación. Esto muestra que el servidor atiende a los cuatro en simultáneo, cada uno en su propio hilo (`ClientHandler`).

```
camila@camila:~/redes-tp1$ cat server.log
INFO - Servidor escuchando en 0.0.0.0:8080
INFO - Nuevo cliente conectado: ('10.0.0.5', 44801)
INFO - Nuevo cliente conectado: ('10.0.0.2', 36434)
INFO - Nuevo cliente conectado: ('10.0.0.3', 51248)
INFO - [('10.0.0.5', 44801)] - Iniciando accion de DOWNLOAD del archivo down2.txtcon SR
INFO - [('10.0.0.2', 36434)] - Iniciando accion de UPLOAD del archivo up1.txtcon SR
INFO - [('10.0.0.2', 36434)] -- Recibiendo el archivo 'up1.txt'...
INFO - [('10.0.0.5', 44801)] -- Enviando archivo 'down2.txt'...
INFO - Nuevo cliente conectado: ('10.0.0.4', 53882)
INFO - [('10.0.0.3', 51248)] - Iniciando accion de UPLOAD del archivo up2.txtcon SR
INFO - [('10.0.0.3', 51248)] -- Recibiendo el archivo 'up2.txt'...
INFO - [('10.0.0.4', 53882)] - Iniciando accion de DOWNLOAD del archivo down1.txtcon SR
INFO - [('10.0.0.4', 53882)] -- Enviando archivo 'down1.txt'...
INFO - [('10.0.0.2', 36434)] -- Archivo 'up1.txt' guardado.
INFO - Cliente ('10.0.0.2', 36434) eliminado
INFO - [('10.0.0.4', 53882)] -- Archivo 'down1.txt' enviado.
INFO - Cliente ('10.0.0.4', 53882) eliminado
INFO - [('10.0.0.5', 44801)] -- Archivo 'down2.txt' enviado.
INFO - Cliente ('10.0.0.5', 44801) eliminado
INFO - [('10.0.0.3', 51248)] -- Archivo 'up2.txt' guardado.
INFO - Cliente ('10.0.0.3', 51248) eliminado
```

Figura 5: Log del servidor atendiendo a los cuatro clientes en simultáneo.

```
camila@camila:~/redes-tp1$    tail -n 3 client1.log client2.log client3.log client4.log
==> client1.log <==
INFO - Preparando subida del archivo: up1.txt
INFO - Conexion establecida - Iniciando transferencia de datos...
INFO - Subida del archivo 'up1.txt' finalizada

==> client2.log <==
WARNING - Timeout en handshake - Timeout esperando ACK al START - Reintento 1/10
INFO - Conexion establecida - Iniciando transferencia de datos...
INFO - Subida del archivo 'up2.txt' finalizada

==> client3.log <==
WARNING - Timeout en handshake - Timeout esperando ACK al START - Reintento 2/10
INFO - Conexion establecida - Iniciando recepción de datos...
INFO - Descarga del archivo 'down1.txt' finalizada - Guardado exitosamente en '/home/camila/redes-tp1/pruebas/'.

==> client4.log <==
INFO - Solicitando descarga del archivo: down2.txt
INFO - Conexion establecida - Iniciando recepción de datos...
INFO - Descarga del archivo 'down2.txt' finalizada - Guardado exitosamente en '/home/camila/redes-tp1/pruebas/'.
```

Figura 6: Los cuatro clientes finalizan sus transferencias.

Por último, la comparación de los hashes MD5 de cada archivo original con su copia confirma que las cuatro transferencias llegaron completas e idénticas a pesar de la pérdida de paquetes.

```
camila@camila:~/redes-tp1$    md5sum pruebas/up1.txt storage/up1.txt pruebas/up2.txt storage/up2.txt storage/down1.txt pruebas/down1.txt storage/down2.txt pruebas/down2.txt
c1ee49b0e5ddf393e0410f9e66e60534  pruebas/up1.txt
c1ee49b0e5ddf393e0410f9e66e60534  storage/up1.txt
2dcfdd6a50ec33a1690c93076b6dc156  pruebas/up2.txt
2dcfdd6a50ec33a1690c93076b6dc156  storage/up2.txt
16c35dd2e7680597f6aa55e6ab41ad3b  storage/down1.txt
16c35dd2e7680597f6aa55e6ab41ad3b  pruebas/down1.txt
b79e47c797003267143f872f0647567f  storage/down2.txt
b79e47c797003267143f872f0647567f  pruebas/down2.txt
```

Figura 7: Hashes MD5 de los archivos originales y sus copias.

5. Preguntas a responder

5.1. Describa la arquitectura Cliente-Servidor

La arquitectura Cliente-Servidor es un modelo de comunicación entre procesos en el que cada uno cumple un rol distinto:

- El servidor es un proceso que está siempre en ejecución, asociado a una dirección IP y un puerto, esperando pedidos. Ofrece un servicio; en este trabajo, almacenar archivos y entregarlos.
- El cliente es el proceso que inicia la comunicación: envía un pedido al servidor y espera la respuesta. Puede conectarse y desconectarse en cualquier momento y no necesita una dirección fija.

Los clientes no se comunican entre sí sino que toda la interacción pasa por el servidor que es quien centraliza el servicio y los datos.

5.1.1. En nuestro trabajo

El servidor se inicia con `start-server` y queda escuchando en una dirección fija. Para eso hace `bind` de su socket (`src/lib/server/server.py`)

```
self.socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
...
self.socket.bind(self.addr)          # por ejemplo ("10.0.0.1", 5000)
```

El cliente en cambio crea su socket sin `bind` (`src/lib/client/client.py`). El sistema operativo le asigna un puerto libre en el primer envío:

```
self.socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
self.server_addr = (host, port)      # solo guarda a dónde enviar
```

Por ejemplo, en una topología de mininet con un servidor y tres clientes:

```
Cliente h2 (10.0.0.2:51234)  ---\
Cliente h3 (10.0.0.3:40021)  ----+--->  Servidor h1 (10.0.0.1:5000)  --->  storage/
Cliente h4 (10.0.0.4:38877)  ---/
```

5.1.2. Servidor concurrente

Un servidor debe poder atender a varios clientes a la vez. En nuestro caso hay un único socket UDP, y el servidor distingue a los clientes por la dirección de origen que devuelve `recvfrom`. Mantiene un diccionario `(ip, puerto): ClientHandler` y deriva cada paquete a la cola del hilo que atiende a ese cliente (`server.py`).

```
data, addr = self.socket.recvfrom(4096)       # paquete de cualquier cliente
with self.lock:
    if addr not in self.clients:              # cliente nuevo: se crea su hilo
        self.clients[addr] = ClientHandler(addr, self.socket, self.storage,
                                           self._remove_client)
        self.clients[addr].recv_data(data)
        self.clients[addr].start()
    else:                                     # cliente conocido: se encola
        self.clients[addr].recv_data(data)
```

Cada `ClientHandler` es un hilo (`threading.Thread`) con su propia cola (`Queue`) y su propia instancia del protocolo. Así, en la prueba de concurrencia (`pruebas/prueba_concu.py`), el servidor atiende al mismo tiempo dos uploads y dos downloads desde cuatro hosts distintos sin que un cliente lento demore a los demás.

5.2. ¿Cuál es la función de un protocolo de capa de aplicación?

El protocolo de la capa de aplicación define cómo se comunican los procesos de una aplicación, es el "idioma" compartido. Este especifica:

- Los tipos de mensajes que se intercambian: pedidos, respuestas, datos, errores.
- La sintaxis de cada mensaje: qué campos tiene, en qué orden y de qué tamaño.
- La semántica de los campos: qué significa cada valor.
- Las reglas de intercambio: cuándo envía cada proceso un mensaje y cómo responde a cada mensaje que recibe, incluidos los casos de error.

Gracias a esta especificación, dos programas escritos de forma independiente pueden operar entre ellos.

5.2.1. En nuestro trabajo

Definimos un protocolo propio con esos cuatro elementos.
Tipos de mensaje (`src/lib/common/constants.py`)

```
TYPE_START = 1     # el cliente pide una operación
TYPE_ACK = 2       # confirmación de recepción
TYPE_CONFIRM = 3   # el cliente confirma que recibió el ACK
TYPE_READY = 4     # el servidor está listo para transferir
TYPE_DATA = 5      # pedazo del archivo
TYPE_ERROR = 6     # la operación no es posible
TYPE_FIN = 7       # no hay más datos
TYPE_FIN_ACK = 8   # confirmación del FIN
```

Sintaxix Un encabezado de 1 byte de tipo y 4 bytes de número de secuencia, seguido por el payload.
Semántica Por ejemplo, el payload de START es el pedido `UPLOAD|gatito.jpg|sr`, y el número de secuencia de un ACK indica qué paquete DATA se confirma.
Reglas Orden START → ACK → CONFIRM → READY, cuándo retransmitir, cómo terminar con FIN y qué hacer ante un error
Además, como usamos UDP, nuestro protocolo de aplicación asume una responsabilidad que normalmente tendría la capa de transporte: garantizar la entrega, mediante Stop & Wait o Selective Repeat.

5.3. Detalle el protocolo de aplicación desarrollado en este trabajo

5.3.1. Formato de los paquetes

Todos los mensajes viajan sobre UDP y tienen la misma estructura: un encabezado de 5 bytes seguido de un payload opcional.

```
byte:  0        1 - 4                   5 ... hasta 2904
      +--------+-----------------------+--------------------------+
      |  tipo  |  numero de secuencia  |  payload (<= 2900 bytes) |
      | 1 byte |  4 bytes, big endian  |                          |
      |        |  con signo            |                          |
      +--------+-----------------------+--------------------------+
```

Hay ocho tipos de mensajes (`constants.py`): START = 1, ACK = 2, CONFIRM = 3, READY = 4, DATA = 5, ERROR = 6, FIN = 7 y FIN_ACK = 8.

Los paquetes se arman y se interpretan con `encode` y `decode` (`src/lib/common/packet_utils.py`)

```
def encode(pack_type, seq_num=NO_SEQ, payload=b""):
    header = pack_type.to_bytes(TYPE_PACKET_SIZE, "big") + seq_num.to_bytes(
        SEQ_NUMB_SIZE, "big", signed=True
    )
    return header + payload

def decode(packet):
    if len(packet) < HEADER_SIZE:
        return None, None, None
    pack_type = int.from_bytes(packet[0:TYPE_PACKET_SIZE], "big")
    seq_num = int.from_bytes(packet[TYPE_PACKET_SIZE:HEADER_SIZE], "big", signed=True)
    payload = packet[HEADER_SIZE:]
    return pack_type, seq_num, payload
```

Los mensajes de control (START, ACK, CONFIRM, READY, ERROR, FIN y FIN_ACK) no llevan un número de secuencia real. Para esos casos, el valor por defecto es `NO_SEQ = -1`.

Los archivos se cortan en pedazos de `PAYLOAD_SIZE = 2900` bytes (`fileHandler.py`), así que un archivo de 5MB son unos 1725 paquetes DATA.

5.3.2. Pedido

El payload del START es un texto con tres campos separados por | (`src/lib/common/request_utils.py`):

```
def build_request(action, filename, protocol):
    return f"{action}{SEPARATOR}{filename}{SEPARATOR}{protocol}".encode()

# build_request("DOWNLOAD", "down5.txt", "sw")  →  b"DOWNLOAD|down5.txt|sw"
```

El servidor lo separa con `parse_request` y verifica que tenga exactamente 3 partes (`REQUEST_SIZE`); si no, lo descarta como formato inválido.

5.3.3. Handshake de 4 mensajes

```
 CLIENTE                                          SERVIDOR
   |--- START "UPLOAD|gatito.jpg|sr" ------------> |  valida el pedido
   |<-------------------------------- ACK ---------|
   |--- CONFIRM ----------------------------------> |  handshake completo
   |<----------------------------- READY x3 -------|  (o ERROR x10 si no se puede)
   |                                               |
   |================ transferencia ================|
   |                                               |
   |--- FIN --------------------------------------> |
   |<-------------------------------- FIN_ACK -----|
```

START → ACK. El cliente reintenta el START cada 2s, hasta 10 veces (`src/lib/client/uploadHandler.py`). Si el servidor recibe un START repetido, porque su ACK se perdió, vuelve a responder ACK.

```
for i in range(HANDSHAKE_RETRIES):
    # Fase 1: START -> ACK
    packet = encode(TYPE_START, payload=request)
    self.client.send_packet(packet)
    if self._wait_for_ack():
        break
    logger.warning(TIMEOUT_ACK.format(num=i + 1,
                                      max_retries=HANDSHAKE_RETRIES))
else:
    return None           # 10 intentos sin ACK: el servidor no responde
```

CONFIRM → READY/ERROR. El CONFIRM le asegura al servidor que el cliente recibió el ACK. El servidor responde READY si la operación es posible. Como los READY no se confirman, se envían 3 separados 50ms. Si se pide un archivo que no existe, el servidor responde con ERROR, repetido 10 veces cada 0,5s (`src/lib/server/downloadHandler.py`). Mientras espera, el cliente reenvía CONFIRM cada 1s, durante hasta 5s.

5.3.4. Transferencia de datos

En UPLOAD el emisor es el cliente y el receptor es el servidor; mientras que en DOWNLOAD es al revés. Además, el mismo código de protocolo se usa en ambos lados porque recibe un "contexto" (`Client` o `ClientHandler`) que sabe enviar y recibir paquetes. Los dos protocolos heredan de `TransmissionProtocol` que define `send_data` y `receive_data` y a su vez implementan el cierre con FIN / FIN_ACK. Tanto `send_data` como `receive_data` devuelven `True` si la transferencia terminó bien y `False` si se abandonó.

Stop & Wait (`src/lib/protocol/stopAndWait.py`)

Se envía un paquete por vez y se espera su ACK hasta `SW_TIMEOUT = 0.07s`. Si no llega, se reenvía el mismo paquete. Si el mismo paquete pasa 10s sin confirmarse (`SW_STALL_TIMEOUT`), el emisor asume que el otro lado ya no responde y abandona.

El receptor confirma todo DATA que recibe, pero solo guarda el que tiene el número esperado. Así, si se pierde un ACK y el emisor reenvía el paquete, el duplicado se vuelve a confirmar pero no se guarda dos veces.

Selective Repeat (`src/lib/protocol/selectiveRepeat.py`)

El emisor puede tener hasta `WINDOW_SIZE = 8` paquetes enviados sin confirmar, y cada uno tiene su propio timer de `SR_TIMEOUT = 0.08s`. Los números que viajan son el número absoluto módulo 17 ($2 \times 8 + 1$). En cada vuelta, el emisor llena la ventana, procesa un ACK si llega y reenvía solo los paquetes vencidos.

Igual que en Stop & Wait, hay un límite de seguridad: si la base de la ventana no avanza durante 10s (`SR_STALL_TIMEOUT`), el emisor abandona y devuelve `False`.

El receptor confirma cada paquete individualmente, guarda en un buffer los que llegan adelantados y los entrega en orden cuando se completa la secuencia. Los que están por debajo de la ventana (duplicados ya entregados) se vuelven a confirmar pero se descartan.

5.3.5. Finalización

El cierre está implementado una sola vez en `TransmissionProtocol` (`transmissionProtocol.py`) y lo usan los dos protocolos. Se usa un tipo de mensaje propio, FIN_ACK, para que la confirmación del cierre no se confunda con el ACK de un DATA.

Emisor Cuando recibió el ACK de todos los paquetes, envía FIN y espera un FIN_ACK durante un timeout. Si no llega, reenvía FIN hasta 10 veces (`MAX_RETRIES`). Devuelve `True` si recibió el FIN_ACK y `False` si se agotaron los intentos.

```
fin_packet = encode(TYPE_FIN)
for cont_retries in range(1, MAX_RETRIES + 1):
    self.ctx.send_packet(fin_packet)
    try:
        ack_packet = self.ctx.get_packet(timeout=self.timeout)
        packet_type, _, _ = decode(ack_packet)
```

```
        if packet_type == TYPE_FIN_ACK:
            return True                             # cierre confirmado
    except TimeoutError:
        continue                                    # reenviar el FIN
return False
```

Receptor Al recibir FIN responde FIN_ACK pero no termina, sigue escuchando durante 2 timeouts. Si el FIN_ACK se perdió, el emisor reenvía el FIN y el receptor vuelve a contestarle.

```
self.ctx.send_packet(encode(TYPE_FIN_ACK))
deadline = time.monotonic() + self.timeout * 2
while time.monotonic() < deadline:
    packet = self.ctx.get_packet(timeout=DATA_LISTENER_TIMEOUT)
    if decode(packet)[0] == TYPE_FIN:             # FIN repetido
        self.ctx.send_packet(encode(TYPE_FIN_ACK))
```

Recién después de recibir el FIN, `receive_data` devuelve `True` y el archivo se escribe en disco: en el storage del servidor si es un upload, o en la carpeta `-d` del cliente si es un download.

Abandono del receptor Si el receptor pasa 5s sin recibir ningún paquete asume que el emisor dejó de responder y `receive_data` devuelve `False`. En ese caso el archivo incompleto no se guarda, lo verifican `client/downloadHandler.py` y `server/uploadHandler.py`.

```
if not protocol.receive_data(data):
    return                      # transferencia incompleta: no se escribe el archivo
file_handler.write(data)
```

5.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

5.4.1. Servicios comunes

Ambos protocolos ofrecen:

- Multiplexación y demultiplexación: mediante los números de puerto, varios procesos de un mismo host pueden usar la red a la vez y cada segmento o datagrama se entrega al proceso correcto.
- Detección de errores: un checksum en el encabezado permite descartar datos corruptos.

5.4.2. TCP (Transmission Control Protocol)

- Orientado a la conexión: antes de transmitir, establece una conexión con un three-way handshake y al final la cierra.
- Entrega confiable y en orden: numera los bytes, confirma lo recibido con ACKs y retransmite ante timeouts o ACKs duplicados. La aplicación recibe los datos completos, sin duplicados y en orden.
- Flujo de bytes: no respeta los límites de los mensajes de la aplicación.
- Control de flujo: el receptor anuncia una ventana de recepción, para que el emisor no le envíe más de lo que puede almacenar.
- Control de congestión: el emisor ajusta su ventana de congestión según las pérdidas detectadas.
- Full-duplex y punto a punto: una conexión une exactamente dos extremos.
- Encabezado de 20 bytes como mínimo.

5.4.3. UDP (User Datagram Protocol)

- No orientado a la conexión: cada datagrama se envía de forma independiente, sin establecimiento previo.
- Sin garantías de entrega: los datagramas pueden perderse, duplicarse o llegar desordenados y no se hace nada al respecto.
- Orientado a mensajes: cada `sendto` produce un datagrama que llega entero o no llega.
- Sin control de flujo ni de congestión: la aplicación decide a qué ritmo envía.
- Encabezado de solo 8 bytes con poca sobrecarga y sin demora de conexión.

5.4.4. Cuándo usar cada uno

TCP es apropiado cuando la información debe llegar completa y en orden y se tolera más demora: navegación web, correo, transferencia de archivos, acceso remoto.

UDP es apropiado cuando importa más la latencia que recuperar cada dato, cuando los intercambios son cortos y una respuesta perdida se puede pedir de nuevo o cuando la aplicación prefiere implementar sus propios mecanismos: streaming y videollamadas, juegos en línea.

5.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión; en caso de tenerlos, descríbalos

El control de congestión busca que los emisores no saturen la red. Consiste en detectar señales de congestión (pérdidas de paquetes o aumento del RTT) y reducir la tasa de envío en respuesta. En nuestro caso, el protocolo no cuenta con control de congestión. En por eso que:

- La ventana de Selective Repeat es fija: es `WINDOW_SIZE = 8` y el emisor siempre intenta tener 8 paquetes en tránsito, sin importar cuántas pérdidas detecte.
- Los timeouts son constantes: son `SW_TIMEOUT = 0.07` y `SR_TIMEOUT = 0.080`. No se estima el RTT ni se agranda el timeout tras retransmisiones sucesivas.
- Una pérdida no cambia el ritmo de envío, solo provoca la retransmisión del paquete afectado.

Lo que sí tiene el protocolo es un límite estático de datos en tránsito: 1 paquete en Stop & Wait y 8 en Selective Repeat. Esto acota cuánta carga puede introducir un emisor, pero no es control de congestión porque no reacciona al estado de la red. De la misma forma, retransmitir solo los paquetes cuyo timer venció evita tráfico innecesario, pero es una propiedad del mecanismo de recuperación de errores. Tampoco son control de congestión los límites de abandono (`SW_STALL_TIMEOUT` y `SR_STALL_TIMEOUT` de 10 s, e `IDLE_TIMEOUT` de 5s en el receptor). Sirven para no quedar esperando para siempre a un extremo que dejó de responder, pero mientras la transferencia avanza no cambian ni la ventana ni el ritmo de envío.

5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

Selective Repeat es más rápido que Stop & Wait, ya que mientras espera ACKs puede seguir enviando, con hasta 8 paquetes en tránsito. Stop & Wait en cambio, envía un paquete y permanece inactivo hasta recibir su ACK, por lo que en cada RTT transmite un solo paquete.

La única excepción es un archivo que entra en un único paquete DATA (hasta 2900 bytes). En este caso ambos protocolos se comportan igual: envían el paquete, esperan su ACK y lo retransmiten si vence el timer. Los tiempos son prácticamente iguales, solo difieren en el valor del timeout (70 ms en Stop & Wait y 80 ms en Selective Repeat), lo que ante una pérdida puede cambiar el resultado en unos pocos milisegundos.

A partir de dos paquetes, Selective Repeat siempre es al menos igual de rápido, aunque la diferencia no es constante:

- Tamaño del archivo: cuantos más paquetes tiene el archivo, mayor es la ventaja. Stop & Wait necesita aproximadamente un RTT por paquete, mientras que Selective Repeat envía hasta 8 por RTT.
- RTT: con un RTT bajo, la espera por cada ACK es casi nula y Stop & Wait pierde poco tiempo por lo que la diferencia se achica. A mayor RTT, mayor la ventaja de Selective Repeat.

- Pérdidas: cada pérdida le cuesta a Stop & Wait un timeout completo con la transferencia detenida mientras que en Selective Repeat el resto de la ventana sigue avanzando. A mayor pérdida, mayor es la diferencia en segundos entre ambos protocolos, aunque la ventaja relativa de Selective Repeat disminuye.

6. Dificultades encontradas

- La ventana deslizante no avanzaba: al principio en Selective Repeat al recibir el ACK del paquete de la base no se movía la ventana por lo que el emisor dejaba de enviar paquetes nuevos y la transferencia se trababa. Se resolvió desplazando la base de la ventana cada vez que se confirma el paquete más antiguo pendiente.
- Emisor en bucle infinito: cuando el receptor abandonaba la transferencia, el emisor seguía retransmitiendo el mismo paquete indefinidamente porque nunca iba a recibir su ACK. Se agregaron los límites `SW_STALL_TIMEOUT` y `SR_STALL_TIMEOUT` por lo que si en 10s la transferencia no avanza el emisor abandona, libera al cliente y deja un error en el log.
- ACK del handshake confundido con el ACK de un DATA: un ACK del handshake que llegaba tarde tenía número de secuencia 0 y podía tomarse como la confirmación del primer DATA. Se cambió el número de secuencia de los mensajes de control a `NO_SEQ = -1`.

7. Conclusión

Desarrollamos una aplicación cliente-servidor de transferencia de archivos sobre UDP que garantiza la entrega en la capa de aplicación. El protocolo diseñado, con un encabezado de tipo y número de secuencia, un handshake de cuatro mensaje y un cierre con FIN y FIN_ACK, permite subir y descargar archivos binarios con dos estrategias de recuperación de errores: Stop & Wait y Selective Repeat.

Las pruebas en Mininet mostraron que los archivos llegan completos e idénticos al original con un 10 % de pérdida, y que el servidor atiende a varios clientes en simultáneo, cada uno en su propio hilo. Se confirmó que Selective Repeat es más rápido que Stop & Wait porque mantiene varios paquetes en tránsito en lugar de esperar el ACK de cada uno.

El trabajo nos permitió ver por qué TCP incorpora cada uno de sus mecanismos: números de secuencia, los ACK y los timers resuelven la pérdida y la duplicación.

Referencias

[1] *Transporte*. Diapositivas de clase, Introducción a los Sistemas Distribuidos (75.43), FIUBA.

[2] *Sockets*. Clase práctica, Introducción a los Sistemas Distribuidos (75.43), FIUBA.

[3] *TCP: Control de Flujo y Control de Congestión*. Diapositivas de clase, Redes (TA048), FIUBA.

[4] *Mininet*. Diapositivas de clase, Introducción a los Sistemas Distribuidos (75.43), FIUBA.